🟢 Project 1 — Animated Hero Section

> **Topics covered:** `gsap.from()`, `gsap.timeline()`, `stagger`, position parameter (`"-="`)
> **Difficulty:** Beginner
> **Files:** `index.html`, `style.css`, `script.js`

🎯 What We Are Building

A full dark-themed landing page **hero section** for a fictional design studio called *Luminary*. When the page loads, every element animates in one by one — the navbar drops down, the heading slides up line by line, the right-side visual card slides in from the right, and floating badges pop in last.

This is the **most common real-world use of GSAP timeline**. Every agency website, SaaS landing, and portfolio uses this exact pattern.

Live animation sequence:

```
Navbar drops in
  → Tag line slides up
    → Heading line 1 slides up
      → Heading line 2 slides up (stagger)
        → Subheading fades in
          → Buttons pop in
            → Stats appear
              → [Right side simultaneously] Card slides in
                → Ring fades in
                  → Badges pop in one by one
```

📁 File Structure

```
project-1-hero-section/
├── index.html
├── style.css
└── script.js
```

🧱 Part 1 — HTML Structure

```html
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
```

```html
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Luminary Studio — Hero Section</title>
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
    <link
      href="https://fonts.googleapis.com/css2?
family=Playfair+Display:wght@700;900&family=DM+Sans:wght@300;400&display=swap"
      rel="stylesheet"
    />
    <link rel="stylesheet" href="style.css" />
  </head>
  <body>

    <nav class="navbar">
      <div class="nav-logo">Luminary.</div>
      <ul class="nav-links">
        <li><a href="#">Work</a></li>
        <li><a href="#">About</a></li>
        <li><a href="#">Services</a></li>
        <li><a href="#">Contact</a></li>
      </ul>
    </nav>

    <section class="hero">

      <div class="hero-content">
        <p class="hero-tag">✦ Creative Design Studio</p>
        <h1 class="hero-heading">
          <span class="line-1">We craft</span>
          <span class="line-2">Bold Visions.</span>
        </h1>
        <p class="hero-sub">
          From brand identity to digital experiences —
          we turn ideas into work that moves people.
        </p>
        <div class="hero-buttons">
          <button class="btn-primary">See Our Work</button>
          <button class="btn-secondary">Let's Talk →</button>
        </div>
        <div class="hero-stats">
          <div class="stat">
            <span class="stat-number">120+</span>
            <span class="stat-label">Projects Done</span>
          </div>
          <div class="stat-divider"></div>
          <div class="stat">
            <span class="stat-number">08</span>
            <span class="stat-label">Years Active</span>
          </div>
          <div class="stat-divider"></div>
          <div class="stat">
            <span class="stat-number">40+</span>
            <span class="stat-label">Happy Clients</span>
```

```
          </div>
        </div>
      </div>

      <div class="hero-visual">
        <div class="visual-card">
          <div class="card-inner">
            <div class="card-circle"></div>
            <div class="card-line"></div>
            <div class="card-line short"></div>
            <div class="card-dot-row">
              <span></span><span></span><span></span>
            </div>
          </div>
        </div>
        <div class="float-badge top">
          <span class="badge-icon">★</span>
          <span class="badge-text">Award Winning</span>
        </div>
        <div class="float-badge bottom">
          <span class="badge-icon">↗</span>
          <span class="badge-text">Global Reach</span>
        </div>
        <div class="deco-ring"></div>
      </div>

    </section>

    <script src="https://cdn.jsdelivr.net/npm/gsap@3.14.2/dist/gsap.min.js">
</script>
    <script src="script.js"></script>
  </body>
</html>
```

💡 Key HTML Decisions Explained

1. Why is the heading split into two `<span>` elements?

```
<h1 class="hero-heading">
  <span class="line-1">We craft</span>
  <span class="line-2">Bold Visions.</span>
</h1>
```

Because we want to animate each line **independently** — line 1 slides in first, then line 2 slides in a fraction of a second later. If we left the heading as a single element, the entire thing would animate as one block. By splitting into spans, GSAP can target `.line-1` and `.line-2` separately and stagger them.

2. Why are elements set to `opacity: 0` in CSS?

```
.hero-tag   { opacity: 0; }
.line-1     { opacity: 0; }
.line-2     { opacity: 0; }
/* ...etc */
```

We hide all elements by default in CSS so that on slow connections — before JavaScript loads — the user doesn't see a flash of all content appearing at once. GSAP will make them visible during the animation. This is called **FOUC prevention** (Flash of Unstyled Content).

3. Why are the two `<script>` tags at the bottom of `<body>`?

```
<script src="https://cdn.jsdelivr.net/npm/gsap@3.14.2/dist/gsap.min.js">
</script>
<script src="script.js"></script>
```

Because scripts at the bottom of `<body>` run **after** all the HTML has been parsed. This means when our `script.js` runs, all the `.hero-tag`, `.navbar`, `.visual-card` elements already exist in the DOM and GSAP can find and animate them. If scripts were in `<head>`, GSAP would try to animate elements that don't exist yet.

🎨 Part 2 — CSS

The CSS Variables Block

```
:root {
  --bg: #0a0a0a;
  --surface: #111111;
  --accent: #e8ff47;
  --accent-dim: rgba(232, 255, 71, 0.12);
  --text-primary: #f0ede6;
  --text-muted: #7a7870;
  --border: rgba(240, 237, 230, 0.08);
  --font-display: "Playfair Display", serif;
  --font-body: "DM Sans", sans-serif;
}
```

Why CSS variables?
All colors and fonts are defined in one place. If you want to change the accent color from yellow-green to blue, you change it in one spot — `--accent: #4af` — and the entire design updates. This is the professional way to manage design tokens.

Hiding Elements Before Animation

```
.navbar       { opacity: 0; }
.hero-tag     { opacity: 0; }
.line-1       { opacity: 0; }
.line-2       { opacity: 0; }
.hero-sub     { opacity: 0; }
.hero-buttons { opacity: 0; }
.hero-stats   { opacity: 0; }
.visual-card  { opacity: 0; }
.deco-ring    { opacity: 0; }
.float-badge  { opacity: 0; }
```

Every element that GSAP will animate is set to `opacity: 0` in CSS. GSAP's `gsap.from()` will animate **from** `opacity: 0` back to whatever the element's natural state is. Since the element is already `opacity: 0` in CSS, the animation starts invisible and reveals it — perfect.

The Decorative Ring CSS Animation

```
.deco-ring {
  animation: spin 18s linear infinite;
}

@keyframes spin {
  from { transform: rotate(0deg); }
  to   { transform: rotate(360deg); }
}
```

The ring's **rotation** is handled by a pure CSS `@keyframes` animation — GSAP only handles making it **visible** (opacity). We use CSS for the rotation because it's a simple infinite loop that never needs to change. GSAP handles complex, one-time, sequenced animations. CSS handles simple infinite loops. Use each for what it's best at.

⚡ Part 3 — JavaScript (GSAP)

This is the core of the project. Let's go through every single line.

Step 1 — Create the Timeline

```
const tl = gsap.timeline({
  defaults: {
    duration: 0.8,
    ease: "power3.out",
  },
});
```

What is `defaults`?

Normally, every `gsap.from()` or `gsap.to()` call needs its own `duration` and `ease`. With `defaults`, you set them once on the timeline and every tween inside **inherits** those values automatically.

Without defaults — repetitive:

```
tl.from(".navbar",   { duration: 0.8, ease: "power3.out", y: -30, opacity: 0
});
tl.from(".hero-tag", { duration: 0.8, ease: "power3.out", y: 20,  opacity: 0
});
tl.from(".hero-sub", { duration: 0.8, ease: "power3.out", y: 30,  opacity: 0
});
```

With defaults — clean:

```
const tl = gsap.timeline({ defaults: { duration: 0.8, ease: "power3.out" }});
tl.from(".navbar",   { y: -30, opacity: 0 });
tl.from(".hero-tag", { y: 20,  opacity: 0 });
tl.from(".hero-sub", { y: 30,  opacity: 0 });
```

Same result — far less code.

What is `power3.out`?

`power3.out` means the animation starts fast and decelerates smoothly at the end — like something sliding into place and stopping gently. It's perfect for UI entrance animations because it feels natural and responsive. The higher the number (power1, power2, power3, power4), the more dramatic the deceleration.

Step 2 — Navbar Animation

```
tl.from(".navbar", {
  y: -30,
  opacity: 0,
  duration: 0.6,
});
```

What happens:

- The navbar starts at `y: -30` (30px above its normal position) and `opacity: 0` (invisible)
- It animates TO its normal position (`y: 0`) and `opacity: 1`

- `duration: 0.6` overrides the default 0.8 for just this tween — the navbar is simple so it can be faster

Why `y: -30` specifically?
The navbar is at the top of the page. Moving it from above (`y: -30`) feels natural — it enters from where it belongs. If we used `y: 30` it would rise up from below, which would feel weird for a top navigation bar.

Step 3 — Tag Line Animation

```
tl.from(".hero-tag", {
  y: 20,
  opacity: 0,
}, "-=0.2");
```

The position parameter `"-=0.2"` — this is new and important.

By default, each tween in a timeline starts **exactly when the previous one ends**. The position parameter lets you shift that timing.

- `"-=0.2"` → start this tween **0.2 seconds before** the previous one ends (overlap)
- `"+=0.5"` → start this tween **0.5 seconds after** the previous one ends (gap)
- `"0"` → start at the very beginning of the timeline
- `"<"` → start at the same time as the previous tween (fully parallel)

Why use overlap here?
Without overlap, each element would fully finish before the next starts — the animation would feel slow and choppy. With small overlaps (`-=0.2`, `-=0.3`), elements start appearing before the previous one has completely landed. This creates a **flowing, fluid feeling** — like a wave washing across the page.

Step 4 — Heading with Stagger

```
tl.from([".line-1", ".line-2"], {
  y: 60,
  opacity: 0,
  stagger: 0.15,
}, "-=0.3");
```

Three things happening here:

1. Array of targets:
`[".line-1", ".line-2"]` — you can pass an array of selectors to target multiple different elements in one tween. GSAP treats them as a group.

2. `y: 60` — larger movement:
The heading is the biggest, most important element on the page. A larger `y` value (60px) creates a more

dramatic entrance, matching its visual importance. Smaller elements like the tag use `y: 20`. The bigger the element, the bigger the entrance.

3. `stagger: 0.15`:
`line-1` starts animating first. `line-2` starts 0.15 seconds later. This creates the famous "words appearing one by one" effect you see on almost every premium website. The timing is short (0.15s) because we want it to feel like a quick reveal, not a slow typewriter.

What it looks like:

```
t=0.00s : "We craft" starts sliding up
t=0.15s : "Bold Visions." starts sliding up
t=0.80s : Both lines have fully settled
```

Step 5 — Subheading

```
tl.from(".hero-sub", {
  y: 30,
  opacity: 0,
}, "-=0.4");
```

Same pattern — slides up, fades in. The `-=0.4` overlap means this starts while the heading is still finishing its animation. On the screen, the user experiences a smooth cascade of content appearing.

Rule of thumb for `y` values:

- Large hero headings → `y: 40-80`
- Medium text (subheadings, tags) → `y: 20-30`
- Small elements (buttons, labels) → `y: 10-20`

The movement distance should be proportional to the element's visual weight.

Step 6 — CTA Buttons

```
tl.from(".hero-buttons", {
  y: 20,
  opacity: 0,
  scale: 0.95,
}, "-=0.3");
```

New property: `scale: 0.95`

The buttons start at 95% of their normal size and grow to 100%. This is barely noticeable on its own — just a 5% scale change. But combined with `opacity` and `y`, it adds a subtle "pop" that makes buttons feel more tangible and clickable.

Try removing `scale: 0.95` and compare. The animation still works, but the buttons feel a little flat. The tiny scale change adds personality.

Step 7 — Stats Row

```
tl.from(".hero-stats", {
  y: 20,
  opacity: 0,
}, "-=0.3");
```

The stats are the last element on the left side. They appear last because they are supplementary information — the user's eye should reach them after absorbing the heading, subheading, and CTA.

Animation hierarchy = visual hierarchy.
The order you animate elements in should match their order of importance on the page.

Step 8 — Visual Card (Right Side)

```
tl.from(".visual-card", {
  x: 60,
  opacity: 0,
  duration: 1,
}, "-=1.2");
```

Two important things here:

1. `x: 60` instead of `y`:
The card is on the RIGHT side of the page. It makes sense for it to enter from the RIGHT (`x: 60` = starts 60px to the right). Left side elements slide up. Right side element slides in from the right. The direction of movement matches where the element lives on the page.

2. `"-=1.2"` — large overlap:
We use `-=1.2` seconds of overlap here. This means the card starts animating while the left side content is still revealing. On screen, both sides of the hero animate simultaneously rather than left-first-then-right. This is what makes the hero feel balanced and cinematic — both halves come alive together.

Step 9 — Decorative Ring

```
tl.from(".deco-ring", {
  opacity: 0,
  scale: 0.6,
  duration: 1,
}, "-=0.6");
```

The ring starts at 60% size and fades in. `scale: 0.6` creates a "growing outward" effect that feels organic — like a ripple expanding. It's a decorative element so its animation is slower (`duration: 1`) and more subtle, meant to be noticed after the main elements.

Step 10 — Floating Badges with Stagger

```
tl.from([".float-badge.top", ".float-badge.bottom"], {
  opacity: 0,
  scale: 0.8,
  duration: 0.5,
  stagger: 0.2,
}, "-=0.4");
```

The two badges appear last — they are decorative details, not primary content. They use `stagger: 0.2` so they don't appear at the exact same time (which would look mechanical). Top badge appears first, bottom badge appears 0.2s later.

`scale: 0.8` → starts at 80% size → grows to 100%. This "pop in from small" is the classic badge/chip reveal animation.

🗺 Complete Timeline Visualization

Here's the entire animation sequence mapped out in time:

```
Time (approx)
──────────────────────────────────────────────────
0.0s  │ Navbar slides down
0.4s  │ Tag line slides up
0.7s  │ "We craft" slides up
0.85s │ "Bold Visions." slides up (stagger)
1.0s  │ Subheading fades in
1.2s  │ Buttons pop in
1.4s  │ Stats appear
──────────────────────────────────────────────────
      │ (Right side starts at the same time as left)
0.2s  │ Visual card slides in from right
0.8s  │ Decorative ring grows in
1.0s  │ Top badge pops in
1.2s  │ Bottom badge pops in (stagger)
```

```
──────────────────────────────────────────────────────────
~1.5s │ Full hero is visible ✓
```

🧪 Experiments for Students

Try these modifications to deepen your understanding:

1. Remove all position parameters (`"-=..."`) and observe:
Each element will wait for the previous one to fully finish before starting. The animation becomes slow and choppy. This shows you why overlaps are important.

2. Change `stagger: 0.15` on the heading to `stagger: 0.6`:
The two heading lines will feel very separated — almost like two separate animations. `0.15` feels like one fluid reveal. `0.6` feels like two separate reveals. Small stagger = flows together. Large stagger = distinct steps.

3. Change `y: -30` on the navbar to `y: 30`:
The navbar will now rise up from below instead of dropping from above. It works — but feels wrong because the navbar visually belongs at the top. Direction of movement should always match the element's position on the page.

4. Try `ease: "elastic"` on the buttons instead of `"power3.out"`:
The buttons will bounce and overshoot when they appear — very playful. `power3.out` is professional. `elastic` is fun. The ease changes the entire personality of the animation.

5. Try adding `repeat: -1, yoyo: true` to the timeline:

```
const tl = gsap.timeline({
  repeat: -1,
  yoyo: true,
  defaults: { duration: 0.8, ease: "power3.out" }
});
```

The entire hero section will play forward, then play backward, then forward again — forever. This is not useful in production but it demonstrates that `repeat` and `yoyo` work on the **entire timeline**, not just individual tweens.

☑ Key Concepts Learned

Concept	Where Used
`gsap.from()`	Every animation — elements animate FROM start values TO current state
`gsap.timeline()`	Wraps all tweens into one sequenced animation
`defaults` on timeline	Sets `duration` and `ease` once for all tweens

Concept	Where Used
`stagger`	Heading lines, floating badges
Position parameter `"-=n"`	Every tween — creates fluid overlapping sequence
`x` vs `y` direction	Card uses `x`, content uses `y` — matches visual position
`scale` on entrance	Buttons and badges — adds tactile "pop" feeling
CSS `opacity: 0`	All animated elements hidden by default until GSAP reveals them